

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS:
Annexure A

Total Marks: 50

Code of Conduct:

I CH.Bhavya Sri/192411249__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

CH.Bhavya Sri

Signature of the student:

AnnexureA

Analytical Problem Solving

1. RSA Encryption Analysis

Given $p=11$, $q=13$

- a. Compute n $\phi(n)$
- b. Choose $e=7$ and find the private key d
- c. Encrypt the message $M=9$
- d. Decrypt the ciphertext and verify the result
- e. Analyze how changing e affects security

2. Block Cipher Mode Evaluation

Given plaintext blocks:

$P=[1010,1010,1111,1010]$, Key = $K=1100$

- a. Encrypt using ECB and CBC modes (Assume IV = 0000)
- b. Compare the ciphertext outputs
- c. Analyze which mode hides patterns better and explain why

3. DES vs 3-DES vs Blowfish Performance Study

Given:

- a. DES key size = 56 bits, time = 2 ms/block
- b. 3-DES key size = 168 bits, time = 6 ms/block
- c. Blowfish key size = 128 bits, time = 1

ms/block For 1000 blocks of data:

- i. Calculate total encryption time for each algorithm
- ii. Analyze trade-offs between security and performance
- iii. Recommend the best algorithm for secure real-time communication

4. Diffie-Hellman Key Exchange Analysis

Given:

- a. $p=23$, $g=5$
- b. User A private key = 6
- c. User B private key = 15
- d. Compute public keys for A and B
- e. Compute the shared secret key
- f. Demonstrate how a Man-in-the-Middle attacker could alter the exchange
- g. Suggest a secure method to prevent the attack

5. ECC vs RSA Comparison

Given:

- a. RSA key size = 1024 bits
- b. ECC key size = 160 bits (equivalent security)
- c. Device processing capacity = limited (IoT device)
- d. Analyze computational cost for both systems
- e. Estimate which consumes less power and memory
- f. Justify which cryptosystem is better for the given scenario with reasoning

Q1. RSA Encryption Analysis

Given:

Prime number $p=11$

Prime number $q=13$

Public exponent $e=7$

Message $M=9$

(a) Compute n and $\phi(n)$

Formula

$$n=p \times q$$

$$\phi(n)=(p-1)(q-1)$$

Calculation

$$n=11 \times 13=143$$

$$\phi(n)=(11-1)(13-1)$$

$$=10 \times 12$$

$$=120 \quad \boxed{=120}$$

Answer

$$n = 143$$

$$\phi(n) = 120$$

(b) Choose $e=7$ and Find the Private Key d

The private key must satisfy

$$e \times d \equiv 1 \pmod{\phi(n)}$$

Substitute the values

$$7 \times d \equiv 1 \pmod{120} \quad \boxed{7 \times d \equiv 1 \pmod{120}}$$

Find the multiplicative inverse of 7 modulo 120.

Try values of d :

$$7 \times 103 = 721$$

Divide by 120

$$721 = 120 \times 6 + 1$$

Therefore,

$$721 \bmod 120 = 1$$

Hence,

$$d = 103$$

Keys

Public Key = (7,143)

Private Key = (103,143)

(c) Encrypt the Message M=9

Encryption formula

$$C = M \bmod n$$

Substitute the values

$$C = 9 \bmod 143$$

Using modular arithmetic

Step 1

$$9^2 = 81$$

Step 2

$$9^4 = 81^2 = 6561 \bmod 143 = 126$$

Step 3

$$9^7 = 9^4 \times 9^2 \times 9$$

Substitute values

$$=126 \times 81 \times 9$$

First multiply

$$126 \times 81 = 10206$$

Take modulus

$$10206 \bmod 143 = 53$$

Multiply by 9

$$53 \times 9 = 477$$

Take modulus

$$477 \bmod 143 = 48$$

Ciphertext

$$C = 48$$

(d) Decrypt the Ciphertext

Decryption formula

$$M = C d \bmod n$$

Substitute the values

$$M = 48 103 \bmod 143$$

Using modular exponentiation,

$$48 103 \bmod 143 = 9$$

Recovered Message

$$M = 9$$

The decrypted message is the same as the original message.

Therefore,

Encryption and decryption are successfully verified.

(e) Analyze How Changing e Affects Security

The public exponent e plays an important role in RSA.

Effects of Choosing Different Values of e

Small value of e

Encryption becomes faster.

Decryption becomes slower.

Very small values (such as 3) may reduce security if proper padding is not used.

Large value of e

Encryption becomes slower.

Provides stronger resistance against certain attacks.

Requires more computation.

The value of e must always satisfy

$$\gcd(e, \phi(n)) = 1$$

A commonly used secure value is

$$e=65537$$

because it provides a good balance between speed and security.

Result

Computed modulus:

$$n=143$$

Euler Totient:

$$\phi(n)=120$$

Public Key:

$$(7, 143)$$

Private Key:

(103,143)

Ciphertext:

48

Decrypted Message:

9

Conclusion: The RSA algorithm successfully encrypts and decrypts the message while ensuring secure communication using public and private keys.

Q2. Block Cipher Mode Evaluation

Given:

Plaintext blocks:

$P = [1010, 1010, 1111, 1010]$

Key:

$K = 1100$

Initialization Vector (IV) = 0000

Assume encryption is performed using the XOR operation.

(a) Encrypt Using ECB Mode

In ECB mode, each plaintext block is encrypted independently.

Formula:

$[C = P \oplus K]$

For Block 1:

```
1010
1100
----
0110
```

For Block 2:

```
1010
1100
----
0110
```

For Block 3:


```
1111
1100
----
0011
```

For Block 4:

```
1010
1100
----
0110
```

Therefore, the ECB ciphertext is:

```
0110
0110
0011
0110
```

Encrypt Using CBC Mode

In CBC mode, each plaintext block is first XORed with the previous ciphertext block before encryption.

Formula:

$$[C_i = (P_i \oplus C_{i-1}) \oplus K]$$

For Block 1:

```
1010
0000
----
1010
1010
1100
----
0110
```

$$C_1 = 0110$$

For Block 2:

```
1010
0110
----
1100
1100
1100
----
0000
```

$$C_2 = 0000$$

For Block 3:

```
1111
0000
----
1111
```

1111
1100

0011

$C_3 = 0011$

For Block 4:

1010
0011

1001
1001
1100

0101

$C_4 = 0101$

Therefore, the CBC ciphertext is:

0110
0000
0011
0101

(b) Compare the Ciphertext Outputs

ECB Ciphertext:

0110
0110
0011
0110

CBC Ciphertext:

0110
0000
0011
0101

In ECB mode, identical plaintext blocks produce identical ciphertext blocks. In CBC mode, identical plaintext blocks produce different ciphertext blocks because each block depends on the previous ciphertext block.

(c) Analyze Which Mode Hides Patterns Better

ECB mode encrypts each block independently. If the same plaintext block appears multiple times, the corresponding ciphertext block is also repeated. This reveals patterns in the encrypted data and makes ECB less secure.

CBC mode combines each plaintext block with the previous ciphertext block before encryption. As a result, even if the plaintext blocks are identical, the ciphertext blocks are different. This hides patterns and provides better security than ECB mode.

Result:

The ECB ciphertext is:

0110
0110
0011
0110

The CBC ciphertext is:

0110
0000
0011
0101

Hence, **CBC mode is more secure than ECB mode because it hides plaintext patterns and provides better confidentiality.**

Q3. DES vs 3-DES vs Blowfish Performance Study

Given:

DES key size = 56 bits, Time = 2 ms/blocks

3-DES key size = 168 bits, Time = 6 ms/block

Blowfish key size = 128 bits, Time = 1 ms/block

Number of data blocks = 1000

(i) Calculate Total Encryption Time for Each Algorithm

The total encryption time is calculated using the formula:

$$\boxed{\text{Total Time} = \text{Time per Block} \times \text{Number of Blocks}}$$

For DES:

$$\begin{aligned} &= 2 \times 1000 \quad \boxed{= 2000 \text{ ms}} \\ &= 2 \text{ seconds} \end{aligned}$$

For 3-DES:

$$\begin{aligned} &= 6 \times 1000 \quad \boxed{= 6000 \text{ ms}} \\ &= 6 \text{ seconds} \end{aligned}$$

For Blowfish:

$$\begin{aligned} &= 1 \times 1000 \quad \boxed{= 1000 \text{ ms}} \\ &\boxed{= 1 \text{ second}} \end{aligned}$$

(ii) Analyze the Trade-offs Between Security and Performance

DES uses a 56-bit key and encrypts data in 2 milliseconds per block. Although it is faster than 3-DES, its key size is too small and can be broken using modern computing techniques. Therefore, it is no longer considered secure.

3-DES uses a 168-bit key and performs the encryption process three times. This provides much stronger security than DES but increases the encryption time to 6 milliseconds per block. It offers better security at the cost of lower performance.

Blowfish uses a 128-bit key and encrypts one block in only 1 millisecond. It provides strong security with high encryption speed, making it suitable for applications that require both security and fast performance.

(iii) Recommend the Best Algorithm for Secure Real-Time Communication

For 1000 data blocks:

DES requires **2000 ms (2 seconds)**.

3-DES requires **6000 ms (6 seconds)**.

Blowfish requires **1000 ms (1 second)**.

Among the three algorithms, Blowfish provides the best balance between security and performance. It has strong encryption, fast execution time, and lower computational cost. Therefore, Blowfish is the most suitable algorithm for secure real-time communication.

Result

Total encryption time for DES = **2000 ms (2 seconds)**

Total encryption time for 3-DES = **6000 ms (6 seconds)**

Total encryption time for Blowfish = **1000 ms (1 second)**

Hence, **Blowfish is recommended for secure real-time communication because it provides strong security with the fastest encryption performance among the given algorithms.**

Q4. Diffie-Hellman Key Exchange Analysis

Given:

- Prime number, $p=23$
- Generator, $g=5$
- User A private key = 6
- User B private key = 15

Q4. Diffie-Hellman Key Exchange Analysis

Given:

$$p = 23$$

$$g = 5$$

$$\text{User A Private Key} = 6$$

$$\text{User B Private Key} = 15$$

(a) Compute the Public Keys of User A and User B

Formula:

$$\text{Public Key} = g^{(\text{Private Key})} \bmod p$$

For User A:

$$A = 5^6 \bmod 23$$

$$A = 15625 \bmod 23$$

$$A = 8$$

Therefore, User A Public Key = **8**

For User B:

$$B = 5^{15} \bmod 23$$

$$B = 30517578125 \bmod 23$$

$$B = 19$$

Therefore, User B Public Key = **19**

(b) Compute the Shared Secret Key

For User A:

$$\text{Shared Key} = 19^6 \bmod 23$$

$$\text{Shared Key} = 2$$

For User B:

$$\text{Shared Key} = 8^{15} \bmod 23$$

$$\text{Shared Key} = 2$$

Therefore, both users obtain the same shared secret key.

Shared Secret Key = 2

(c) Demonstrate How a Man-in-the-Middle (MITM) Attacker Could Alter the Exchange

A Man-in-the-Middle attacker intercepts the public keys exchanged between User A and User B. The attacker replaces the original public keys with fake public keys. User A establishes a secret key with the attacker instead of User B, and User B also establishes a different secret key with the attacker. The attacker can then read, modify, and forward all messages without either user knowing.

(d) Suggest a Secure Method to Prevent the Attack

The attack can be prevented by using Digital Signatures, Digital Certificates, Public Key Infrastructure (PKI), or SSL/TLS. These methods authenticate the users before the key exchange and ensure that the exchanged public keys are genuine.

Result

User A Public Key = 8

User B Public Key = 19

Shared Secret Key = 2

Hence, the Diffie-Hellman algorithm successfully generates a common secret key, and authentication techniques protect it from Man-in-the-Middle attacks.

Q5. ECC vs RSA Comparison

Given:

RSA Key Size = 1024 bits

ECC Key Size = 160 bits (Equivalent Security)

Device Processing Capacity = Limited (IoT Device)

CO2_Assessment 1

(a) Analyze the Computational Cost for Both Systems

RSA uses a 1024-bit key and performs encryption and decryption using large mathematical calculations. These operations require more processor time and higher computational power.

ECC uses a 160-bit key to provide the same level of security as a 1024-bit RSA key. Since the key size is much smaller, ECC performs calculations faster and requires less processing power.

Therefore, ECC has a lower computational cost than RSA.

(b) Estimate Which Consumes Less Power and Memory

RSA uses larger keys, so it requires more memory to store keys and more power to perform encryption and decryption operations.

ECC uses smaller keys, so it requires less memory and consumes less battery power. This makes ECC suitable for devices with limited resources.

Therefore, ECC consumes less power and memory than RSA.

(c) Justify Which Cryptosystem is Better for the Given Scenario

The given device is an IoT device with limited processing capacity. Such devices require fast execution, low memory usage, and low power consumption.

RSA provides strong security but requires larger keys and higher computational resources. This increases execution time and power consumption.

ECC provides the same level of security using much smaller keys. It performs encryption and decryption faster, requires less memory, and consumes less power.

Hence, ECC is more suitable for IoT devices because it is efficient and provides strong security with lower resource requirements.

(d) Analyze the Computational Cost for Both Systems

RSA uses a 1024-bit key and performs encryption and decryption using large mathematical operations. These calculations require more processing time and higher computational power.

ECC uses a 160-bit key to provide the same level of security. Since the key size is much smaller, the calculations are completed faster and require less processing power.

Therefore, ECC has a lower computational cost than RSA.

(e) Estimate Which Consumes Less Power and Memory

RSA requires larger keys and more computational resources. As a result, it consumes more memory and battery power.

ECC uses smaller keys and fewer computational resources. Therefore, it requires less memory and consumes less power.

Hence, ECC consumes less power and memory than RSA.

(f) Justify Which Cryptosystem is Better for the Given Scenario

The given scenario involves an IoT device with limited processing capacity. Such devices require fast execution, low memory usage, and low power consumption.

RSA provides strong security but requires larger keys, more memory, and higher computational power.

ECC provides the same level of security with much smaller keys. It performs encryption and decryption faster, uses less memory, and consumes less battery power.

Therefore, ECC is the better cryptosystem for the given scenario because it is more efficient and well suited for IoT devices while maintaining strong security.

Result

RSA Key Size = **1024 bits**

ECC Key Size = **160 bits**

ECC requires less computational cost than RSA.

ECC consumes less memory and power than RSA.

Therefore, **ECC is the better cryptosystem for IoT devices because it provides equivalent security with faster execution, lower memory usage, and lower power consumption.**